

CS2040 Visualgo quizzes questions and sols

sorting, hard, no time

Potential Pivots using Quicksort

1.

An array of 11 integers is being sorted in non-decreasing order using **Quicksort**. Suppose the algorithm has just finished the first pass of partitioning and pivot swapping thus changing the content of the original array into the following array:
[102,105,100,106,110,113,109,114,111,112,107]
From the resulting array above, determine how many integers could have been the pivot?
Note: elements == pivot are partitioned to the right.

1

Enter a number

You answered this question correctly! :)

An array of 10 integers is being sorted in non-decreasing order using **Quicksort**. Suppose the algorithm has just finished the first pass of partitioning and pivot swapping thus changing the content of the original array into the following array:
[101,103,105,107,106,111,113,112,110,108]
From the resulting array above, determine how many integers could have been the pivot?
Note: elements == pivot are partitioned to the right.

1

Enter a number

You answered this question wrongly.

The correct answer is: 3

How many **pass(es)** is/are required to sort an array of n=5 integers: [26, 24, 31, 10, 39] using this version of **Bubble Sort**?

```
j = 0;
do {
    swapped = false; ++j; // each time this line is executed is counted as one pass
    for (i = 0; i < n-j; ++i)
        if (A[i] > A[i+1]) { // each execution of this if-statement is counted as one comparison
            swap(A[i], A[i+1]);
            swapped = true;
        }
} while (swapped);
```

3

Enter a number

You answered this question wrongly.

The correct answer is: 4
Not sure what went wrong? [Click here to try out the visualization.](#)

**Remember that bubble loop with early stopping will still traverse the sorted array once before stopping"

swaps in bbbble sort, comparisons

How many **comparison(s)** is/are required to sort an array of n=6 integers: [23, 24, 12, 4, 13, 19] using this version of **Bubble Sort**?

```
j = 0;
do {
    swapped = false; ++j; // each time this line is executed is counted as one pass
    for (i = 0; i < n-j; ++i)
        if (A[i] > A[i+1]) { // each execution of this if-statement is counted as one comparison
            swap(A[i], A[i+1]);
            swapped = true;
        }
} while (swapped);
```

14

Enter a number

You answered this question correctly! :)

Comparison(not swap) of bubble sort: 5+4+3+2=14
Basically it is (n-1) + (n-2) + (n-3)+... +(n-k) , where k is the number of passes.
Be careful of the contribution of 2: the last part is traversing through a sorted array with 2 comparisons

9.

Rearrange the integers to produce the content of the array A=[**39,10,7,34,20,29,5,31**] with n integers after 7 passes of this version of **Bubble Sort**.

```
for (j = 0; j < 7; ++j) // 7 passes
    for (i = 0; i < n-j-1; ++i)
        if (A[i] > A[i+1])
            swap(A[i], A[i+1]);
```

5

7

10

20

29

31

34

39

You answered this question correctly! :)

1.

An array of 11 integers is being sorted in non-decreasing order using **Quicksort**. Suppose the algorithm has just finished the first pass of partitioning and pivot swapping thus changing the content of the original array into the following array:
[102,105,100,106,110,113,109,114,111,112,107]
From the resulting array above, determine how many integers could have been the pivot?
Note: elements == pivot are partitioned to the right.

1

Enter a number

You answered this question correctly! :)

An array of 10 integers is being sorted in non-decreasing order using **Quicksort**. Suppose the algorithm has just finished the first pass of partitioning and pivot swapping thus changing the content of the original array into the following array:
[101,103,105,107,106,111,113,112,110,108]
From the resulting array above, determine how many integers could have been the pivot?
Note: elements == pivot are partitioned to the right.

1

Enter a number

You answered this question wrongly.

The correct answer is: 3

2.

How many **swap(s)** is/are required to sort an array of n=5 integers: [5,20,7,23,16] using this version of **Bubble Sort**?

```
for (j = 0; j < n-1; ++j)
    for (i = 0; i < n-j-1; ++i)
        if (A[i] > A[i+1])
            swap(A[i], A[i+1]);
```

2

Enter a number

You answered this question wrongly.

The correct answer is: 3

5.

How many **swap(s)** is/are required to sort an array of n=5 integers: [35,11,7,8,15] using this version of **Bubble Sort**?

```
for (j = 0; j < n-1; ++j)
    for (i = 0; i < n-j-1; ++i)
        if (A[i] > A[i+1])
            swap(A[i], A[i+1]);
```

6

Enter a number

You answered this question correctly! :)

6.

How many **comparison(s)** is/are required to sort an array of n=6 integers: [17,18,16,13,14,15] using this version of **Insertion Sort**?

```
for (i = 1; i < n; ++i) {
    e = A[i]; j = i;
    while (j > 0) {
        if (A[j-1] > e) // each execution of this if-statement is counted as one comparison
            A[j] = A[j-1];
        else
            break;
        j--;
    }
    A[j] = e;
}
```

14

Enter a number

You answered this question correctly! :)

be careful that not all comparison will be executed, if the while loop is broken early

7.

Rearrange the integers to produce the content of the array A=[**2, 4, 37,10,21, 34, 38**] with n integers after 4 passes of this version of **Selection Sort**.

```
our_min = i;
for (j = i+1; j < n; ++j)
    if (A[j] < A[our_min])
        our_min = j;
swap(A[i], A[our_min]);
```

2

4

10

21

37

34

38

You answered this question correctly! :)

4.

Rearrange the integers to produce the content of the array A=[**39,36,24,18,31,34**] with n integers after 5 passes of this version of **Bubble Sort**.

```
for (j = 0; j < 5; ++j) // 5 passes
    for (i = 0; i < n-j-1; ++i)
        if (A[i] > A[i+1])
            swap(A[i], A[i+1]);
```

18

24

31

34

36

39

You answered this question correctly! :)

Becareful that insertion sort is not bubble sort. Best to use pen and paper to figure out first before answering the quiz (because the web ui interface let you swap adjacent numbers like bubble sorting)

Tips: you can see that the insertion sort is guaranteed to produce sorted array after n-1 passes. Why?

Linked List

This kind of question is actually solvable
You can identify potential pivot by first compare the array with the final sorted array
The potential pivot must be at the same position in these 2 arrays
after that, check that for each potential pivot, all num in front <(no == given question) pivot, all num>= pivot are at right.

passes in bubble sort

8.

How many **swap(s)** is/are required to sort an array of n=5 integers: [33,12,35,13,27] using this version of **Bubble Sort**?

```
for (j = 0; j < n-1; ++j)
    for (i = 0; i < n-j-1; ++i)
        if (A[i] > A[i+1])
            swap(A[i], A[i+1]);
```

5

Enter a number

You answered this question correctly! :)

7.

Rearrange the integers to produce the content of the array A=[**2, 4, 37,10,21, 34, 38**] with n integers after 4 passes of this version of **Selection Sort**.

```
our_min = i;
for (j = i+1; j < n; ++j)
    if (A[j] < A[our_min])
        our_min = j;
swap(A[i], A[our_min]);
```

2

4

10

21

37

34

38

You answered this question correctly! :)

4.

Rearrange the integers to produce the content of the array A=[**39,36,24,18,31,34**] with n integers after 5 passes of this version of **Bubble Sort**.

```
for (j = 0; j < 5; ++j) // 5 passes
    for (i = 0; i < n-j-1; ++i)
        if (A[i] > A[i+1])
            swap(A[i], A[i+1]);
```

18

24

31

34

36

39

You answered this question correctly! :)

Becareful that bubble sort might change "number to be elevated" midway. Tips: you can see

3.

In the pseudocode program below, `list` is an initially empty Singly Linked List. The function `populateList()` adds the integers [7, 2, 1, 9] to the tail of `list` sequentially. What is the output of the program?
Select 'No Answer' if the program results in an error (e.g., accessing an empty list).

```
populateList() {
    int sum = 0;
    Node n = list.head; // list.head/list.tail points to the first/last integer in list
    n = list.head;
    sum += n.value;
    sum += n.next.value;
    sum += n.next.value;
    sum += n.next.value;
    print sum;
}
```

No answer

11

Enter a number

You answered this question correctly! :)

Note that accessing `n.next.value` will not change `n`'s pointer.

7.

How many **pass(es)** is/are required to sort an array of n=5 integers: [22, 18, 12, 11, 17] using this version of **Bubble Sort**?

```
j = 0;
do {
    swapped = false; ++j; // each time this line is executed is counted as one pass
    for (i = 0; i < n-j; ++i)
        if (A[i] > A[i+1]) { // each execution of this if-statement is counted as one comparison
            swap(A[i], A[i+1]);
            swapped = true;
        }
} while (swapped);
```

4

Enter a number

You answered this question correctly! :)

3.

How many **comparison(s)** is/are required to sort an array of n=5 integers: [15, 17, 16, 13, 14] using this version of **Insertion Sort**?

```
for (i = 1; i < n; ++i) {
    e = A[i]; j = i;
    while (j > 0) {
        if (A[j-1] > e) // each execution of this if-statement is counted as one comparison
            A[j] = A[j-1];
        else
            break;
        j--;
    }
    A[j] = e;
}
```

10

Enter a number

You answered this question correctly! :)

Insertion comparison
for each element, find the com = (number of element in front that is larger than it + 1)
then sum all com for all elements

that the bubble sort is guaranteed to produce sorted array after n-1 passes. Why?

7.

In the pseudocode program below, `list` is an initially empty Singly Linked List. The function `populateList()` adds the integers [8, 1, 2] to the tail of `list` sequentially. What is the output of the program?
Select 'No Answer' if the program results in an error (e.g., accessing an empty list).

```
populateList() {
    int sum = 0;
    Node n = list.head; // list.head/list.tail points to the first/last integer in list
    sum += n.next.value;
    n = n.next;
    sum += n.next.value;
    n = list.head;
    print sum;
}
```

No answer

Enter a number

You answered this question wrongly.

The correct answer is: 3

The Singly Linked List below represents a queue q where q.front() = 61. Then, we perform these operations:

```
q.dequeue();
q.dequeue();
q.enqueue(72);
q.dequeue();
q.enqueue(5);
q.dequeue();
q.enqueue(11);
q.dequeue();
```

What is the output of q.peek() after all these operations above are performed?
Select 'No Answer' if the program results in an error (e.g., accessing an empty list).

☐ No answer

Enter a number

You answered this question wrongly.

The correct answer is: **81**

Not sure what went wrong? [Click here to try out the visualization](#)



5. Suppose you have a quadratic probe hash table of size $M = 11$ with the hash function $h(v) = v \% M$. Insert the keys 39, 11, 17, 28 into the table (in order). Is this the correct resulting hash table after insertion: [11, NULL, NULL, NULL, NULL, NULL, 39, 17, NULL, NULL, 28]?

☐ Yes
☒ No

You answered this question wrongly.

The correct answer is: **Yes**

Not sure what went wrong? [Click here to try out the visualization](#)

10. Suppose you have a quadratic probe hash table of size $M = 11$ with the hash function $h(v) = v \% M$. Click the sequence of integers that represent the probe sequence if you delete 62 from the following hash table:

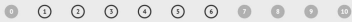
What is the output of q.peek() after all these operations above are performed?
Select 'No Answer' if the program results in an error (e.g., accessing an empty list).

☐ Yes
☒ No

You answered this question wrongly.

The correct answer is: **81**

Not sure what went wrong? [Click here to try out the visualization](#)



Binary Heap

3. What is the **MAXimum** number of **swaps** between heap elements required to construct a max heap of 9 elements using the $O(n)$ BuildHeap(array)?

Enter a number

You answered this question wrongly.

The correct answer is: **7**

don't just blindly use 8-2. Think of the number of involved internal nodes in 9 nodes.

Oh I see, pay attention to the 3 internal nodes (not including root) and root, for these 4 nodes, we pick the maximum length path of swapping to get max swaps.

Minimum comparison/ swaps

9. What is the **MINimum** number of **comparisons** between heap elements required to construct a max heap of 12 elements using the $O(n)$ BuildHeap(array)?

Enter a number

You answered this question correctly! :)

Be careful to choose the minimum comparisons!

One special case(if question doesn't mention it's invalid) is just use the ascending array for the creation of the heap. In that case, no swap and number of comparison is just number of edges (why?)

(u only need to compare along each edge once in that case)

3. In the pseudocode program below, list is an initially empty Singly Linked List. The function populateList() adds the integers [4, 6, 4, 1, 2] to the tail of list sequentially. What is the output of the program?

```
populateList();
int sum = 0;
Node n = list.head; // list.head/list.tail points to the first/last integer in list
n = n.next;
sum += n.next.value;
sum += n.next.value;
sum += n.value;
n = n.next;
sum += n.value;
n = list.head;
print sum;
```

☐ No answer

Enter a number

You answered this question wrongly.

The correct answer is: **22**

Now I see why, it is quadratic

2. What is the hash value of $v = \text{"JKC"}$ if we use the standard hash function for strings and the hash table size $M = 41$? To avoid ambiguity, the C++ version of $h(v)$ is this one:

```
int hash_function(string v) { // assumption 1: v uses ['A'..'Z'] only
    int sum = 0; // assumption 2: v is a short string
    for (auto& c : v) // for each character c in v
        sum = ((sum*26)%M + (c-'A'+1)%M); // M is table size
    return sum;
}
```

Enter a number

You answered this question wrongly.

The correct answer is: **38**

Please restudy the [concept of hashing a string](#)

5. What is the **MAXimum** number of **comparisons** between heap elements required to construct a max heap of 10 elements using the $O(n)$ BuildHeap(array)?

Enter a number

You answered this question wrongly.

The correct answer is: **15**

Similar to reasoning above, just that you will need to compare both sides (2 comparisons) before making a swap.

4. What is the **MAXimum** number of **swaps** between heap elements required to construct a max heap of 10 elements using the $O(n)$ BuildHeap(array)?

Enter a number

You answered this question correctly! :)

n	Min comparisons	Max comparisons	Max swaps
8	7	11	7
9	8	14	7
10	9	15	8
11	10	16	8
12	11	18	10
13	12	20	10
14	13	21	11
15	14	22	11
16	15	26	15

elements left after extraction.

You are given a queue l containing the first few positive integers in ascending order: (front) 1, 2, ..., 6 (back). You want to reorder the content of queue l into another queue o: (front) 2, 4, 6, 5, 3, 1 (back). Note that you can only dequeue an element from l and enqueue an element into o. To help you with this process, you are allowed to temporarily store 0, 1, or some values in exactly one auxiliary stack. Can you do it?

Example 1: You can reorder l = 1, 2, 3 into o = 3, 2, 1 by dequeuing 1, 2, 3 from l in that order into the auxiliary stack, then popping them from the stack in reverse order to get 3, 2, 1 and send them to o.
Example 2: You can reorder l = 1, 2, 3 into o = 1, 5, 2 by dequeuing 1 from l and immediately send 1 to o, then use the auxiliary stack to reverse 2, 3 into 5, 2 before sending them to o.
Example 3: You cannot reorder l = 1, 2, 3 into o = 3, 1, 2 given the constraints.

☒ Yes
☐ No

You answered this question wrongly.

The correct answer is: **Yes**

the key is to enqueue the even number into queue while storing the odd in stack, after that,

Letter	Order Number	Letter	Order Number
A	1	N	14
B	2	O	15
C	3	P	16
D	4	Q	17
E	5	R	18
F	6	S	19
G	7	T	20
H	8	U	21
I	9	V	22
J	10	W	23
K	11	X	24
L	12	Y	25
M	13	Z	26

6. What is the **MAXimum** number of **comparisons** between heap elements required to construct a max heap of 12 elements using the $O(n)$ BuildHeap(array)?

Enter a number

You answered this question correctly! :)

pop and enqueue the odd elements
You are given a queue l containing the first few positive integers in ascending order: (front) 1, 2, ..., 7 (back). You want to reorder the content of queue l into another queue o: (front) 1, 5, 2, 3, 4, 6, 7 (back). Note that you can only dequeue an element from l and enqueue an element into o. To help you with this process, you are allowed to temporarily store 0, 1, or some values in exactly one auxiliary stack. Can you do it?

Example 1: You can reorder l = 1, 2, 3 into o = 3, 2, 1 by dequeuing 1, 2, 3 from l in that order into the auxiliary stack, then popping them from the stack in reverse order to get 3, 2, 1 and send them to o.
Example 2: You can reorder l = 1, 2, 3 into o = 1, 5, 2 by dequeuing 1 from l and immediately send 1 to o, then use the auxiliary stack to reverse 2, 3 into 5, 2 before sending them to o.
Example 3: You cannot reorder l = 1, 2, 3 into o = 5, 1, 2 given the constraints.

☒ No
☐ Yes

hash

10. Consider a linear probe hash table of size $M = 12$ with the hash function $h(v) = v \% M$. Click the sequence for inserting keys 24, 34, 39, 44, 49, 52, 53, 55, 58 such that the resulting content of the hash table is:

Your answer is: 44, 34, 58, 49, 24, 39, 52, 53, 55
You answered this question correctly! :)



note that this is not a unique sol. the placement of 39, 52, 53, 55, won't matter. What matters is 46 before 34, followed by 58. A caveat is that even though 24 has a unique hash value of 1, it cannot at position 1 because 58 has already occupied before it, so 58 must be before 24

7. Suppose you have a quadratic probe hash table of size $M = 13$ with the hash function $h(v) = v \% M$. Insert the keys 65, 56, 30, 43, 17, 26 into the table (in order). Is this the correct resulting hash table after insertion: [65, 26, NULL, NULL, 56, NULL, NULL, 17, 43, NULL, 30, NULL, NULL]?

☒ No
☐ Yes

You answered this question correctly! :)

10. What is the **MAXimum** number of **swaps** between heap elements required to construct a max heap of 10 elements using the $O(n)$ BuildHeap(array)?

Enter a number

You answered this question correctly! :)

What is the **MAXimum** number of **swaps** between heap elements required to construct a max heap of 11 elements using the $O(n)$ BuildHeap(array)?

Enter a number

You answered this question wrongly.

The correct answer is: **8**

What is the **MAXimum** number of **swaps** between heap elements required to construct a max heap of 12 elements using the $O(n)$ BuildHeap(array)?

Enter a number

You answered this question wrongly.

The correct answer is: **10**

Binary Search Tree(BST)

- deletion is NOT Always Commutative, Counter Example: Simply create four vertices 1, 2, 3, 4 where 2 is the root, 1 is the left subtree of 2, 4 is the right subtree of 2, and 3 is the left subtree of 4. Deleting 1 first and then 2 is different than deleting 2 first (replace 2 with it's successor 3) and then 1.
- The insert operation in BST is NOT (highlighted is false) always not commutative.

Regardless of the initial heap, the max heap is always extracting max. So if there is n initially, after k top elements are extracted, it will be left with n-k least elements.

Consider the leaves first

UFDS

- Assumption: Path compression and union-by-rank heuristics are used
- when findSet(x) is called, the UFDS data structure will change if x is at least 2 edge from root
- Given n elements/disjoint sets at the beginning, the maximum h that can be obtained after a series of union and find is $\text{floor}(\log_2(N))$.
 - to achieve max height, don't call findSet and union symmetrically
- example:
- Given $n = 935$ disjoint sets initially in a UFDS, is it possible to call `unionSet(1, 935)` and/or `findSet(1)` operations to get a single tree with actual height 10 that represents a certain set? Both path-compression and union-by-rank heuristics are used.
- A: FALSE. TO GET height 10 you will need $\text{min pow}(2, 10) = 1024$ sets
- note that there is no max number of sets as you can always compress path to achieve arbitrary low height